

# Fundamentals of C Programming

Introduces C programming structure, data types, input/output, operators, control statements, arrays, pointers, strings and functions for solving real-world problems.

Course Code: 3045

Category: Program Core

Credits: No Credit

L:T:P = 4:0:0

Programming Theory

COURSE OVERVIEW

## Fundamentals of C Programming (3045)

This is a standalone digital textbook, revision guide, workbook and answer guide prepared from the uploaded official syllabus PDF.

## Student-friendly summary

Introduces C programming structure, data types, input/output, operators, control statements, arrays, pointers, strings and functions for solving real-world problems.

**Study tip:** Read module-wise separate topics thoroughly. Definitions, diagrams, formulas/procedures, and expected answers English technical terms must be practised thoroughly.

| Course Metadata        |                                                                  |
|------------------------|------------------------------------------------------------------|
| ITEM                   | DETAILS                                                          |
| Course Code            | 3045                                                             |
| Base Course Code       | 3045                                                             |
| Course Title           | Fundamentals of C Programming                                    |
| Programme / Department | Diploma in Electronics / Electronics & Communication Engineering |

| ITEM               | DETAILS            |
|--------------------|--------------------|
| Semester           | 3                  |
| Course Category    | Program Core       |
| Credits            | No Credit          |
| L:T:P              | 4:0:0              |
| Periods / semester | 60                 |
| Subject Type       | Programming Theory |

## Course Objectives

- Provide basic knowledge in C programming.
- Use control blocks, looping, string manipulation and functions to solve real world problems.
- Utilize C programming for real-life programming tasks.

## Prerequisite Knowledge

- Basic principles and theorems of Engineering Mathematics - Mathematics I & II
- Computer, operating system, internet applications and basic Python programming - Introduction to IT Systems Lab

## Course Outcomes

| CO  | DESCRIPTION                                                      | HOURS | LEVEL    |
|-----|------------------------------------------------------------------|-------|----------|
| CO1 | Make use of programming concepts with C programming.             | 13    | Applying |
| CO2 | Make use of iterative control structures and arrays in programs. | 15    | Applying |
| CO3 | Develop programs using pointers and strings.                     | 15    | Applying |
| CO4 | Make use of functions to solve problems effectively.             | 15    | Applying |

## Module Map

1. **Module 1:** C programming basics (13 h)
2. **Module 2:** Control structures and arrays (15 h)
3. **Module 3:** Pointers and strings (15 h)
4. **Module 4:** Functions (15 h)

## How to study this subject

- Understand syntax using small examples.
- Trace every program line by line.
- Practise algorithms before coding.
- Run at least two test cases for each program.
- Revise common compiler errors.

## Exam preparation method

- Write algorithm first, then program.
- Use correct indentation and comments.
- Mention sample input/output.
- Check loops, arrays, pointers and functions carefully.

## Quick Navigation Cards

### KEY POINTS BANK

## Key Points Bank - Fundamentals of C Programming

Quick reference cards for revision. Use this only after reading the module explanation.

## Program structure

**Header -> main() -> declarations -> statements -> return**

**Meaning:** Standard C program layout.

**Where used:** All programming answers.

**Common mistake:** Writing statements outside functions.

## Input/output

**scanf("%d", &x); printf("%d", x);**

**Meaning:** Read and display values.

**Where used:** Basic programs.

**Common mistake:** Forgetting & in scanf for normal variables.

## if-else

## **if(condition){...} else {...}**

**Meaning:** Decision making.

**Where used:** Flow-control problems.

**Common mistake:** Using = instead of ==.

## **for loop**

### **for(init; condition; update){...}**

**Meaning:** Count-controlled repetition.

**Where used:** Series/array problems.

**Common mistake:** Wrong loop boundary.

## **Array**

### **int a[10];**

**Meaning:** Stores same-type values in continuous memory.

**Where used:** List/matrix problems.

**Common mistake:** Accessing a[10] in 10-size array.

## **Pointer**

### **int \*p = &x**

**Meaning:** Stores address of a variable.

**Where used:** Pointers and call by reference.

**Common mistake:** Dereferencing uninitialised pointer.

## **String functions**

### **strlen, strcpy, strcat, strcmp**

**Meaning:** Built-in string operations.

**Where used:** String handling.

**Common mistake:** Not including string.h.

# Function

```
return_type name(parameters){...}
```

**Meaning:** Reusable block of code.

**Where used:** Modular programs.

**Common mistake:** Mismatch between prototype and definition.

MODULE 1 • 13 HOURS

## C programming basics

Use C program structure, keywords, variables, constants, data types, I/O, operators and type casting.

## Learning outcome

Use C program structure, keywords, variables, constants, data types, I/O, operators and type casting.

**Malayalam support:** പദപരിചയം meaning പദപരിചയം, പദപരിചയം English technical terms പദപരിചയം answer പദപരിചയം.

## Important terms

main()

keyword

variable

constant

data type

scanf

printf

operator

type casting

## Official module outcome table

| SYLLABUS CODE | MODULE OUTCOME                                                     | HOUR S | LEVEL          |
|---------------|--------------------------------------------------------------------|--------|----------------|
| M1.01         | Understand structure of a C program                                | 1      | Understandi ng |
| M1.02         | Use keywords, variables, constants, data types and type qualifiers | 3      | Applying       |
| M1.03         | Experiment with basic input and output functions                   | 3      | Applying       |

| SYLLABUS CODE | MODULE OUTCOME                                                                      | HOURS | LEVEL    |
|---------------|-------------------------------------------------------------------------------------|-------|----------|
| M1.04         | Use arithmetic, relational, conditional, logical, bit-wise and assignment operators | 4     | Applying |
| M1.05         | Apply type casting on data types                                                    | 2     | Applying |

## Topic-wise explanation

**Structure of a C program:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Keywords, variables and constants:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Data types and type qualifiers:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Output and input functions:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Arithmetic operators:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Relational and logical operators:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

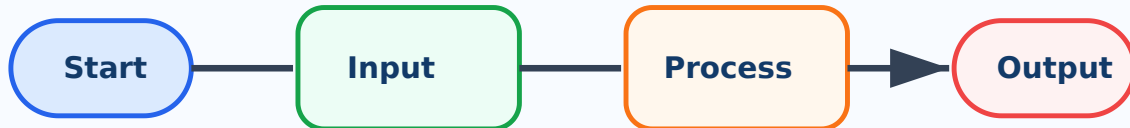
**Increment/decrement and conditional operators:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Assignment and bitwise operators:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

writing code.

**Type casting:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

## Visual block / diagram idea



C programming basics: algorithm writing model

Use this type of labelled diagram/block in answers when applicable.

## Worked / practice example

```
#include <stdio.h>
int main(void){
    int a,b;
    scanf("%d%d", &a, &b);
    printf("Sum = %d", a+b);
    return 0;
}
```

Explain header file, main function, variable declaration, input, processing and output.

## Common mistakes

- Missing semicolon or braces.
- Using wrong format specifier in scanf/printf.
- Array index out of range.
- Using uninitialised pointer or variable.
- Not showing sample input and output.

## Expected Questions

**M1-Q1** Explain the main concept of C programming basics with syntax or format.

**M1-Q2** Write a C program / code fragment related to Structure of a C program.

**M1-Q3** Compare two important concepts from C programming basics.

**M1-Q4** Find and correct common errors in a C program using C programming basics.

**M1-Q5** Write the algorithm and output for a problem from C programming basics.

**M1-Q6** List applications and exam tips for C programming basics.

## Module checklist

- ✓ Syntax and keywords understood.
- ✓ Program can be traced manually.
- ✓ At least two sample inputs tested.
- ✓ Common errors checked.
- ✓ Output is written exactly.

### MODULE 2 • 15 HOURS

## Control structures and arrays

Use decision statements, loops, unconditional control statements, one-dimensional arrays, two-dimensional arrays and matrix operations.

## Learning outcome

Use decision statements, loops, unconditional control statements, one-dimensional arrays, two-dimensional arrays and matrix operations.

**Malayalam support:** പദങ്ങൾ meaning പദപരമ്പരകൾ, പദങ്ങൾ English technical terms പദപരമ്പരകൾ answer പദങ്ങൾ.

## Important terms

if-else

switch

while

do-while

for

break

continue

array



## Official module outcome table

| SYLLABUS CODE | MODULE OUTCOME                                         | HOURS | LEVEL    |
|---------------|--------------------------------------------------------|-------|----------|
| M2.01         | Experiment with control structures                     | 2     | Applying |
| M2.02         | Make use of looping structures                         | 2     | Applying |
| M2.03         | Develop programs with unconditional control statements | 2     | Applying |
| M2.04         | Experiment with one-dimensional array operations       | 3     | Applying |
| M2.05         | Make use of two-dimensional array declaration          | 3     | Applying |
| M2.06         | Make use of matrix operations                          | 3     | Applying |

## Topic-wise explanation

**if, if-else and nested if:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**switch statement:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**while, do-while and for loops:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Nested loops:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

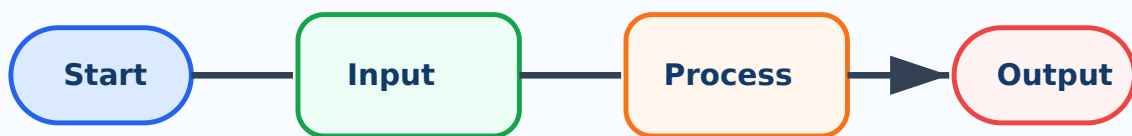
**break, continue and goto:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**1D array insertion, deletion, searching and sorting:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**2D array declaration:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Matrix addition and multiplication:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

## Visual block / diagram idea



Control structures and arrays: algorithm writing model

Use this type of labelled diagram/block in answers when applicable.

## Worked / practice example

```
#include <stdio.h>
int main(void){
    int a,b;
    scanf("%d%d", &a, &b);
    printf("Sum = %d", a+b);
    return 0;
}
```

Explain header file, main function, variable declaration, input, processing and output.

## Common mistakes

- Missing semicolon or braces.
- Using wrong format specifier in scanf/printf.
- Array index out of range.
- Using uninitialised pointer or variable.
- Not showing sample input and output.

## Expected Questions

**M2-Q1** Explain the main concept of Control structures and arrays with syntax or format.

**M2-Q2** Write a C program / code fragment related to if, if-else and nested if.

**M2-Q3** Compare two important concepts from Control structures and arrays.

**M2-Q4** Find and correct common errors in a C program using Control structures and arrays.

**M2-Q5** Write the algorithm and output for a problem from Control structures and arrays.

**M2-Q6** List applications and exam tips for Control structures and arrays.

## Module checklist

- ✓ Syntax and keywords understood.
- ✓ Program can be traced manually.
- ✓ At least two sample inputs tested.
- ✓ Common errors checked.
- ✓ Output is written exactly.

### MODULE 3 • 15 HOURS

## Pointers and strings

Use pointers, pointer arithmetic, strings using pointers, gets/puts and built-in string handling functions.

## Learning outcome

Use pointers, pointer arithmetic, strings using pointers, gets/puts and built-in string handling functions.

**Malayalam support:** പാഠ്യപുസ്തകത്തിൽ ഉൾപ്പെട്ട ഏതെങ്കിലും പ്രശ്നത്തിന്റെ മറുപടി എഴുതുക. English technical terms ഉൾപ്പെടെ.

## Important terms

pointer

address

dereference

pointer arithmetic

string

gets

puts

strlen

strcpy

strcat

strcmp

## Official module outcome table

| SYLLABUS CODE | MODULE OUTCOME                                        | HOURS | LEVEL    |
|---------------|-------------------------------------------------------|-------|----------|
| M3.01         | Develop concepts of pointers in programming           | 2     | Applying |
| M3.02         | Use pointer arithmetic operations                     | 3     | Applying |
| M3.03         | Develop programs for accessing strings using pointers | 3     | Applying |
| M3.04         | Experiment with gets() and puts() functions           | 3     | Applying |
| M3.05         | Use built-in string handling functions                | 4     | Applying |

## Topic-wise explanation

**Basics of pointers:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Pointer arithmetic addition, subtraction, increment and decrement:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

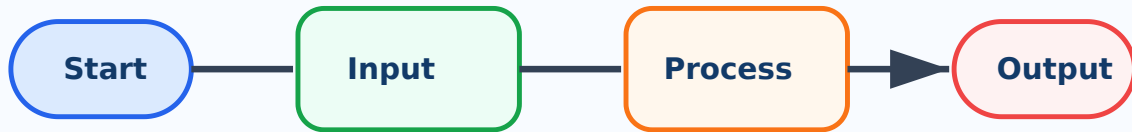
**Pointer comparison:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Accessing strings using pointers:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**gets() and puts() functions:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**String handling functions:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

## Visual block / diagram idea



Pointers and strings: algorithm writing model

Use this type of labelled diagram/block in answers when applicable.

## Worked / practice example

```
#include <stdio.h>
int main(void){
    int a,b;
    scanf("%d%d", &a, &b);
    printf("Sum = %d", a+b);
    return 0;
}
```

Explain header file, main function, variable declaration, input, processing and output.

## Common mistakes

- Missing semicolon or braces.
- Using wrong format specifier in scanf/printf.
- Array index out of range.
- Using uninitialised pointer or variable.
- Not showing sample input and output.

## Expected Questions

**M3-Q1** Explain the main concept of Pointers and strings with syntax or format.

**M3-Q2** Write a C program / code fragment related to Basics of pointers.

**M3-Q3** Compare two important concepts from Pointers and strings.

**M3-Q4** Find and correct common errors in a C program using Pointers and strings.

**M3-Q5** Write the algorithm and output for a problem from Pointers and strings.

**M3-Q6** List applications and exam tips for Pointers and strings.

## Module checklist

- ✓ Syntax and keywords understood.
- ✓ Program can be traced manually.
- ✓ At least two sample inputs tested.
- ✓ Common errors checked.
- ✓ Output is written exactly.

### MODULE 4 • 15 HOURS

## Functions

Define functions, write user-defined function structure, use function declarations/calls/arguments/return type and recursion.

## Learning outcome

Define functions, write user-defined function structure, use function declarations/calls/arguments/return type and recursion.

**Malayalam support:** ഉത്തരം meaning ഉത്തരം/ഉത്തരങ്ങൾ, ഉത്തരം English technical terms ഉത്തരം answer ഉത്തരം.

## Important terms

function

prototype

argument

return type

call by value

call by reference

recursion

## Official module outcome table

| SYLLABUS CODE | MODULE OUTCOME                                            | HOURS | LEVEL         |
|---------------|-----------------------------------------------------------|-------|---------------|
| M4.01         | Outline functions, types and advantages                   | 2     | Understanding |
| M4.02         | Illustrate structure of user-defined functions            | 3     | Understanding |
| M4.03         | Use function declaration, call, arguments and return type | 3     | Applying      |
| M4.04         | Construct different types of functions                    | 2     | Applying      |
| M4.05         | Build programs with call by value and call by reference   | 3     | Applying      |
| M4.06         | Use recursive functions                                   | 2     | Applying      |

## Topic-wise explanation

**Definition of functions:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Structure of user-defined functions:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Function prototype and function call:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Arguments and return type:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

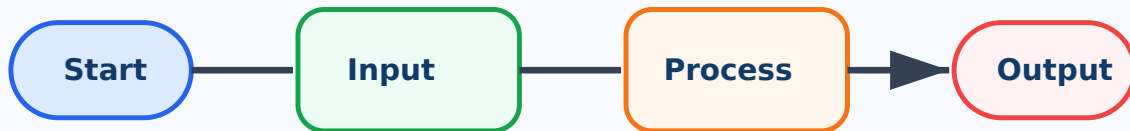
**Types of functions:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Call by value:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Call by reference:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

**Recursive functions:** Learn the syntax first, then write small programs and test with sample inputs. Keep variable names meaningful and explain the program flow before writing code.

## Visual block / diagram idea



Functions: algorithm writing model

Use this type of labelled diagram/block in answers when applicable.

## Worked / practice example

```
#include <stdio.h>
int main(void){
    int a,b;
    scanf("%d%d", &a, &b);
    printf("Sum = %d", a+b);
    return 0;
}
```

Explain header file, main function, variable declaration, input, processing and output.

## Common mistakes

- Missing semicolon or braces.
- Using wrong format specifier in scanf/printf.
- Array index out of range.
- Using uninitialised pointer or variable.
- Not showing sample input and output.

## Expected Questions

**M4-Q1** Explain the main concept of Functions with syntax or format.

**M4-Q2** Write a C program / code fragment related to Definition of functions.



**M4-Q3** Compare two important concepts from Functions.

**M4-Q4** Find and correct common errors in a C program using Functions.

**M4-Q5** Write the algorithm and output for a problem from Functions.

**M4-Q6** List applications and exam tips for Functions.

## Module checklist

- ✓ Syntax and keywords understood.
- ✓ Program can be traced manually.
- ✓ At least two sample inputs tested.
- ✓ Common errors checked.
- ✓ Output is written exactly.

### QUICK REVISION

## One-page Revision Summary

Use this page in the last 24 hours before exam or lab evaluation.

## Module-wise must-know points

1. **Module 1 - C programming basics:** Revise program skeleton, data types, I/O and operator precedence.
2. **Module 2 - Control structures and arrays:** Practise flow control syntax, loop tracing, array operations and matrix programs.
3. **Module 3 - Pointers and strings:** Know address/dereference, pointer arithmetic rules and string functions with examples.
4. **Module 4 - Functions:** Practise function syntax, argument passing and recursive program tracing.

## Important definitions / keywords

keyword

variable

constant

data type

type qualifier

operator

type casting

control statement

loop

array

pointer

string

function

## Important formulas / key points

- Header -> main() -> declarations -> statements -> return
- `scanf("%d", &x); printf("%d", x);`
- `if(condition){...} else {...}`
- `for(init; condition; update){...}`
- `int a[10];`
- `int *p = &x`
- `strlen, strcpy, strcat, strcmp`
- `return_type name(parameters){...}`

## Common exam mistakes

- Missing semicolon or braces.
- Using wrong format specifier in `scanf/printf`.
- Array index out of range.
- Using uninitialised pointer or variable.
- Not showing sample input and output.

## Last-minute checklist

- ✓ Write algorithm first, then program.
- ✓ Use correct indentation and comments.
- ✓ Mention sample input/output.
- ✓ Check loops, arrays, pointers and functions carefully.

## Diagram / table list

- C program structure diagram
- Decision flowchart
- Loop flowchart
- 1D/2D array memory representation
- Pointer/address diagram
- Function call flow
- Recursion stack concept

## Master Answer Key - matched with Expected Questions

Every answer below matches the module expected questions exactly. Attempt first, then verify here.

### M1-Q1 - Explain the main concept of C programming basics with syntax or format.

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for C programming basics, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### M1-Q2 - Write a C program / code fragment related to Structure of a C program.

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for C programming basics, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### M1-Q3 - Compare two important concepts from C programming basics.

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for C programming basics, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### M1-Q4 - Find and correct common errors in a C program using C programming basics.

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for C programming basics, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

## **M1-Q5 - Write the algorithm and output for a problem from C programming basics.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for C programming basics, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

## **M1-Q6 - List applications and exam tips for C programming basics.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for C programming basics, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

## **M2-Q1 - Explain the main concept of Control structures and arrays with syntax or format.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Control structures and arrays, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

## **M2-Q2 - Write a C program / code fragment related to if, if-else and nested if.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Control structures and arrays, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

## **M2-Q3 - Compare two important concepts from Control structures and arrays.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Control structures and arrays, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

## **M2-Q4 - Find and correct common errors in a C program using Control structures and arrays.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Control structures and arrays, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### **M2-Q5 - Write the algorithm and output for a problem from Control structures and arrays.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Control structures and arrays, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### **M2-Q6 - List applications and exam tips for Control structures and arrays.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Control structures and arrays, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### **M3-Q1 - Explain the main concept of Pointers and strings with syntax or format.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Pointers and strings, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### **M3-Q2 - Write a C program / code fragment related to Basics of pointers.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Pointers and strings, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### **M3-Q3 - Compare two important concepts from Pointers and strings.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Pointers and strings, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output ->**

**explanation.** Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### **M3-Q4 - Find and correct common errors in a C program using Pointers and strings.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Pointers and strings, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output ->**

**explanation.** Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### **M3-Q5 - Write the algorithm and output for a problem from Pointers and strings.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Pointers and strings, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output ->**

**explanation.** Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### **M3-Q6 - List applications and exam tips for Pointers and strings.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Pointers and strings, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output ->**

**explanation.** Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### **M4-Q1 - Explain the main concept of Functions with syntax or format.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Functions, and display output clearly.

For exam answers, write **algorithm -> program -> sample input/output -> explanation.**

Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### **M4-Q2 - Write a C program / code fragment related to Definition of functions.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Functions, and display output clearly.

For exam answers, write **algorithm -> program -> sample input/output -> explanation.**

Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

### **M4-Q3 - Compare two important concepts from Functions.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Functions, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

#### **M4-Q4 - Find and correct common errors in a C program using Functions.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Functions, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

#### **M4-Q5 - Write the algorithm and output for a problem from Functions.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Functions, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.

#### **M4-Q6 - List applications and exam tips for Functions.**

Start with the required header files and `main()`. Declare variables with correct data types, read inputs using suitable input statements, process the logic for Functions, and display output clearly. For exam answers, write **algorithm -> program -> sample input/output -> explanation**. Mention common errors such as missing semicolon, wrong format specifier, uninitialised variable, array index out of range and pointer misuse.